

HMAS

ARMONÍA MUSICAL

H.M.A.S. Armonía Musical

Sistema de Gestión Académica y
Administrativa

Stack: Node.js · Express · React · SQLite

Fecha: 8 de julio de 2026

Versión: 1.0.0

Índice

1 Semana 1: Diagnóstico

1.1 Visitas a la organización

1.2 Entrevistas con responsables

1.3 Observación de procesos

1.4 Encuestas a usuarios

1.5 Documentación del problema

2 Semana 2: Análisis y Diseño

2.1 Recopilación de requerimientos

2.2 Diseño de base de datos

2.3 Creación de mockups/interfaces

2.4 Selección de tecnologías

2.5 Documento de diseño

3 Semana 3: Desarrollo — Parte 1

3.1 Configuración del entorno

3.2 Estructura del proyecto

3.3 Creación de base de datos

3.4 Página de inicio

3.5 Formularios básicos

4 Semana 4: Desarrollo — Parte 2

4.1 Registro de datos

4.2 Visualización de datos

4.3 Edición de registros

4.4 Eliminación de datos

4.5 Búsquedas básicas

5 Semana 5: Desarrollo — Parte 3

5.1 Inicio de sesión

5.2 Roles de usuarios

5.3 Reportes

5.4 Diseño responsivo

5.5 Pruebas funcionales

A Anexo A: Diagramas del Usuario Final

A.1 Diagrama de casos de uso

A.2 Diagrama de flujo de usuario

► Visitas a la organización

Se realizaron visitas presenciales a las instalaciones de la **H.M.A.S. Armonía Musical**, ubicada en la zona centro de la ciudad. La escuela cuenta con **3 aulas de clases** equipadas con instrumentos musicales, una **sala de ensayos** con aislamiento acústico, y una **oficina administrativa** donde laboran el director, coordinador, secretaria y tesorero. Se identificó que todos los procesos se llevan a cabo de forma manual, sin ningún sistema informático que respalde la gestión académica ni administrativa.

La estructura organizacional de la institución es la siguiente:

```
graph TD
    D["D[Director  
Martín Reyes]"] --> CA["CA[Coordinación Académica]"]
    D --> AD["AD[Administración]"]
    CA --> P1["P1[Profesores Niveles 1-3]"]
    CA --> P2["P2[Profesores Niveles 4-6]"]
    CA --> P3["P3[Profesores Niveles 7-9]"]
    AD --> SE["SE[Secretaría]"]
    AD --> TE["TE[Tesorería]"]
    P1 --> E1["E1[Estudiantes]"]
    P2 --> E2["E2[Estudiantes]"]
    P3 --> E3["E3[Estudiantes]"]
    style D fill:#c9a84c,color:#0a0a14,stroke:#c9a84c,stroke-width:2px
    style CA fill:#1a2a5e,color:#fff,stroke:#c9a84c,stroke-width:2px
    style AD fill:#1a2a5e,color:#fff,stroke:#c9a84c,stroke-width:2px
```

Figura 1.1 — Organigrama de H.M.A.S. Armonía Musical

► Entrevistas con responsables

Se entrevistaron a los principales responsables de la institución para comprender sus funciones, los procesos que realizan y las principales dificultades que enfrentan en su día a día. A continuación se presentan los perfiles y hallazgos principales:

Rol	Nombre	Hallazgos principales
Director	Martín Reyes	Requiere reportes académicos y financieros para toma de decisiones; actualmente no recibe información consolidada.
Coordinador	Ana López	Evalúa el progreso de los estudiantes con expedientes físicos; carece de un sistema para dar seguimiento individualizado.
Secretaría	Carmen Vargas	Realiza inscripciones en cuadernos físicos; atiende consultas de horarios y pagos de forma verbal, sin registros digitales.
Tesorero	Pedro Mendoza	Lleva la contabilidad en libretas; emite recibos manuscritos; no puede consultar el historial de pagos de forma eficiente.
Profesor	Luis Campos	Toma asistencia en hojas de papel; registra evaluaciones en cuadernos; no cuenta con un sistema para reportar calificaciones.

Tabla 1.1 — Entrevistas con responsables de la organización

► Observación de procesos

Se observó el flujo de trabajo actual (AS-IS) que siguen los estudiantes y el personal administrativo desde que un alumno llega

a la institución hasta que concluye su ciclo. El proceso es completamente manual y carece de integración entre las áreas involucradas.

flowchart TD

```
A[Estudiante llega a la escuela] --> B[Solicita información]
B --> C[Secretaria realiza entrevista]
C --> D[Registra datos en cuaderno]
D --> E[Asigna nivel y horario]
E --> F[Tesorero cobra inscripción en efectivo]
F --> G{Emite recibo?}
G -->|Sí| H[Recibo manuscrito]
G -->|No| I[Sin comprobante]
H --> J[Comienzan clases]
I --> J
J --> K[Profesor toma asistencia en papel]
K --> L[Pago mensual en efectivo]
L --> M[Tesorero registra en libreta]
M --> N[Profesor evalúa en cuaderno]
N --> O[Coordinador elabora reporte manual]
O --> P[Fin]
style O fill:#e74c3c,color:#fff,stroke:#c0392b,stroke-width:2px
```

Figura 1.2 — Diagrama de flujo AS-IS del proceso actual. El paso de reporte manual se resalta en rojo por ser el punto crítico.

Se identificó que el paso de **elaboración de reportes manuales** (resaltado en rojo) es el cuello de botella crítico del proceso, ya que el coordinador debe recopilar información dispersa de cuadernos, libretas y hojas sueltas para generar reportes que frecuentemente llegan tarde o contienen errores.

► Encuestas a usuarios

Se aplicaron encuestas a **50 estudiantes** y **10 miembros del personal** para cuantificar el nivel de satisfacción con los procesos

actuales. Los resultados obtenidos evidencian la necesidad urgente de un sistema digital:

Indicador	Resultado	Interpretación
Estudiantes insatisfechos con el proceso de inscripción	73%	El proceso manual es lento y propenso a errores; los estudiantes esperan largas filas.
Estudiantes que no pueden consultar su estado de pago	68%	No existe un medio para que los alumnos verifiquen si están al día con sus mensualidades.
Personal que considera necesario un sistema digital	80%	La mayoría del personal reconoce que la digitalización mejoraría su productividad.
Estudiantes que han perdido recibos de pago	60%	Los recibos en papel se extravían fácilmente, generando conflictos con tesorería.

Tabla 1.2 — Resultados de encuestas a usuarios

► Documentación del problema

A partir de las visitas, entrevistas, observaciones y encuestas, se documentaron los cinco problemas fundamentales que afectan a la institución:

- ◆ **Registro manual lento:** Las inscripciones se realizan en cuadernos físicos, lo que provoca demoras en la atención a los estudiantes y pérdida de información.

◆ **Pagos sin control:** No existe un sistema de registro de pagos que permita conocer el estado de cuenta de cada estudiante en tiempo real.

◆ **Asistencia en papel:** Los profesores registran la asistencia en hojas sueltas que se extravían o se dañan con facilidad.

◆ **Sin reportes gerenciales:** El director y coordinador no reciben información consolidada para la toma de decisiones estratégicas.

◆ **Progreso sin seguimiento:** No hay un mecanismo para evaluar y dar seguimiento al avance académico de los estudiantes a lo largo de los niveles.

A continuación se presenta un diagrama de causa-efecto (Ishikawa) que analiza las causas raíz que originan la ineficiencia de los procesos:

```

flowchart LR
    subgraph Personas
        A1[Secretaria saturada]
        A2[Tesorero proceso manual]
    end
    subgraph Metodos
        B1[Inscripción en cuaderno]
        B2[Pagos en efectivo sin respaldo]
    end
    subgraph Tecnologia
        C1[Sin software de gestión]
        C2[Archivos físicos sin respaldo]
    end
    subgraph Medicion
        D1[Sin reportes periódicos]
        D2[Sin KPIs ni indicadores]
    end
    CENTER[Procesos Ineficientes]
    A1 --> CENTER
    A2 --> CENTER
    B1 --> CENTER
    B2 --> CENTER
    C1 --> CENTER
    C2 --> CENTER
    D1 --> CENTER
    D2 --> CENTER
    style CENTER fill:#c9a84c,color:#0a0a14,stroke:#c9a84c

```

Figura 1.3 — Diagrama de causa-efecto (Ishikawa) de procesos ineficientes

► Recopilación de requerimientos

Se realizaron sesiones de trabajo con el personal de la institución para definir los requerimientos funcionales y no funcionales del sistema. A continuación se presentan las tablas consolidadas:

Código	Requerimiento funcional	Prioridad
RF001	El sistema debe permitir el registro de nuevos estudiantes con datos personales y nivel asignado.	Alta
RF002	El sistema debe permitir el inicio de sesión con roles diferenciados (admin, estudiante).	Alta
RF003	El sistema debe permitir registrar pagos y consultar el estado de cuenta de cada estudiante.	Alta
RF004	El sistema debe permitir registrar la asistencia diaria de los estudiantes.	Alta
RF005	El sistema debe mostrar el progreso académico del estudiante por nivel.	Media
RF006	El sistema debe generar reportes de ingresos, asistencia y rendimiento académico.	Media
RF007	El sistema debe permitir la edición y eliminación de registros de estudiantes.	Media
RF008	El sistema debe mostrar un dashboard con indicadores clave para el	Media

Código	Requerimiento funcional	Prioridad
	administrador.	
RF009	El sistema debe permitir el registro de usuarios con diferentes roles.	Alta
RF010	El sistema debe contar con un sistema de búsqueda y filtrado de estudiantes y pagos.	Baja

Tabla 2.1 — Requerimientos funcionales del sistema

Código	Requerimiento no funcional	Prioridad
RNF001	La aplicación debe ser responsiva y funcional en dispositivos móviles y de escritorio.	Alta
RNF002	El sistema debe utilizar autenticación segura mediante tokens JWT y contraseñas encriptadas.	Alta
RNF003	La base de datos debe garantizar la integridad referencial mediante claves foráneas.	Alta
RNF004	La interfaz debe cargar los datos en menos de 3 segundos en conexiones de banda ancha.	Media
RNF005	El código fuente debe estar documentado y seguir buenas prácticas de desarrollo.	Media

Tabla 2.2 — Requerimientos no funcionales del sistema

► Diseño de base de datos

Se diseñó un modelo entidad-relación con **7 tablas** que cubren todos los módulos del sistema: usuarios, estudiantes, niveles, pagos, progreso, asistencia y mensajes. El diseño garantiza la integridad referencial mediante claves foráneas y relaciones bien definidas.

```

erDiagram
    users ||--o{ estudiantes : "tiene"
    estudiantes ||--o{ pagos : "realiza"
    estudiantes ||--o{ progreso : "registra"
    estudiantes ||--o{ asistencia : "genera"
    niveles ||--o{ estudiantes : "clasifica"
    niveles ||--o{ pagos : "corresponde"

    users {
        int id PK
        string nombre
        string correo
        string password
        string rol
    }

    estudiantes {
        int id PK
        int usuario_id FK
        string telefono
        string nivel_actual
        string estado
    }

    niveles {
        int id PK
        string nombre
        int duracion_dias
    }

    pagos {
        int id PK
        int estudiante_id FK
        string nivel
        float monto
        string estado
    }

```

```

progreso {
    int id PK
    int estudiante_id FK
    string nivel
    int porcentaje
    string estado
}

asistencia {
    int id PK
    int estudiante_id FK
    date fecha
    boolean presente
}

mensajes {
    int id PK
    string titulo
    string contenido
    string tipo
}

```

Figura 2.1 — Diagrama entidad-relación de la base de datos

Las tablas **pagos**, **progreso** y **asistencia** dependen directamente del estudiante, lo que permite tener un historial completo de la trayectoria académica y financiera. La tabla **niveles** sirve como catálogo para estandarizar la duración y los requisitos de cada etapa formativa.

► Creación de mockups/interfaces

Se elaboraron mockups de las pantallas principales del sistema, definiendo la navegación y los componentes clave de cada vista. El diseño prioriza la experiencia de usuario con una navegación clara y accesible.

```

flowchart TD
    Home[Home] --> Login[Login]
    Login --> SD[StudentDashboard]
    Login --> AD[AdminDashboard]

    SD --> PC[ProfileCard]
    SD --> PB[ProgressBar]
    SD --> PH[PaymentHistory]
    SD --> AC[AttendanceCard]

    AD --> ST[StudentsTable]
    AD --> PP[PaymentsPanel]
    AD --> AP[AttendancePanel]
    AD --> SC[StatsCharts]

    Home --> REG[Register]
    REG --> SD

    ST -->|Editar| ST
    PP -->|Filtrar| PP
    AP -->|Registrar| AP

    style Home fill:#c9a84c,color:#0a0a14,stroke:#c9a84c
    style Login fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style SD fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style AD fill:#1a2a5e,color:#fff,stroke:#c9a84c

```

Figura 2.2 — Diagrama de navegación entre pantallas del sistema

Las pantallas identificadas incluyen: **Home** (página de inicio con presentación institucional), **Login/Register** (autenticación), **StudentDashboard** (perfil, barra de progreso, historial de pagos y asistencia), y **AdminDashboard** (tabla de estudiantes, panel de pagos, panel de asistencia y gráficas estadísticas).

► Selección de tecnologías

Se evaluaron diferentes alternativas tecnológicas para cada capa del sistema, considerando factores como curva de aprendizaje, comunidad, rendimiento y compatibilidad con los requerimientos del proyecto.

Área	Tecnología seleccionada	Alternativas evaluadas	Justificación
Backend	Node.js + Express	Python/Django, PHP/Laravel, Java/Spring	Node.js ofrece alto rendimiento asíncrono, gran ecosistema de paquetes (npm) y curva de aprendizaje rápida. Express es minimalista y flexible para APIs REST.
Frontend	React	Vue.js, Angular, Svelte	React tiene la comunidad más grande, componentes reutilizables y un ecosistema maduro. Ideal para interfaces dinámicas con estado complejo.
Base de datos	SQLite	MySQL, PostgreSQL, MongoDB	SQLite es embebida, no requiere configuración

Área	Tecnología seleccionada	Alternativas evaluadas	Justificación
			de servidor, ideal para proyectos pequeños y medianos. Portabilidad total del archivo .db.
CSS Framework	Bootstrap 5	Tailwind CSS, Material-UI, Bulma	Bootstrap proporciona componentes listos para usar, grid responsivo y amplia documentación. Acelera el desarrollo de interfaces profesionales.
Autenticación	JWT + bcrypt	OAuth 2.0, Passport.js, Session-based	JWT permite autenticación stateless ideal para APIs REST. bcrypt proporciona hashing seguro de contraseñas.

Tabla 2.3 — Evaluación y selección de tecnologías

► Documento de diseño

La arquitectura del sistema sigue un patrón de **3 capas** (Frontend, API, Backend/Database) con una separación clara de responsabilidades. El flujo de datos comienza en el navegador del usuario, pasa por la API Express que aplica middleware de autenticación y finalmente llega a la base de datos SQLite.

```

graph LR
    subgraph Cliente
        BR[Navegador]
    end
    subgraph Frontend
        REACT[React + Bootstrap]
    end
    subgraph API
        EXP[Express + JWT]
        CORS[CORS]
        ENV[dotenv]
    end
    subgraph Backend
        NODE[Node.js]
        BCRYPT[bcrypt]
    end
    subgraph Datos
        DB[SQLite]
    end

    USUARIO[Usuario] --> BR
    BR --> REACT
    REACT --> |Peticiones HTTP| EXP
    EXP --> CORS
    EXP --> ENV
    EXP --> NODE
    NODE --> BCRYPT
    NODE --> DB

    style USUARIO fill:#c9a84c,color:#0a0a14,stroke:#c9a84c
    style BR fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style REACT fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style EXP fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style NODE fill:#1a2a5e,color:#fff,stroke:#c9a84c
    style DB fill:#1a2a5e,color:#fff,stroke:#c9a84c

```

Figura 2.3 — Diagrama de arquitectura del sistema

El flujo de información sigue la siguiente secuencia: (1) el usuario interactúa con el navegador, (2) React renderiza la interfaz y envía peticiones HTTP a la API, (3) Express procesa las rutas aplicando middleware de autenticación (JWT) y configuración (CORS, dotenv), (4) Node.js ejecuta la lógica de negocio con bcrypt para operaciones sensibles, y (5) SQLite almacena y recupera los datos de forma persistente.

► Configuración del entorno

Se configuró el entorno de desarrollo con Node.js v18 LTS y npm. El servidor Express se creó con las dependencias necesarias para manejar CORS, variables de entorno, autenticación y conexión a la base de datos. A continuación se muestra el archivo principal del servidor y las dependencias del proyecto.

```
// server.js
const express = require('express');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Servir frontend en producción
app.use(express.static(path.join(__dirname, '../frontend/dist')));

// Rutas API
app.use('/api/auth', require('./routes/auth'));
app.use('/api/students', require('./routes/students'));
app.use('/api/admin', require('./routes/admin'));

// Inicio del servidor
app.listen(PORT, () => {
  console.log(`Servidor corriendo en puerto ${PORT}`);
});
```

Código 3.1 — Configuración del servidor Express (server.js)

```
{
  "name": "hmars-backend",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "better-sqlite3": "^9.4.3",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "express-validator": "^7.0.1"
  }
}
```

Código 3.2 — Dependencias del backend (package.json)

► Estructura del proyecto

El proyecto se organizó en dos directorios principales (backend y frontend) con una separación clara de responsabilidades. La base de datos se encuentra en un directorio independiente con el script de inicialización.

```
hmars-armonia-musical/ ├── backend/ | | ├──
config/ | | └─ database.js | | ├── middleware/ |
| | ├── auth.js | | └─ admin.js | | ├── routes/ | |
| | ├── auth.js | | ├── students.js | | └─ admin.js
| | ├── server.js | | ├── package.json | | └─ .env └─
frontend/ | | ├── src/ | | ├── App.jsx | | ├──
main.jsx | | ├── pages/ | | ├── Home.jsx | | ├──
| | ├── Login.jsx | | ├── Register.jsx | | ├──
| | ├── StudentDashboard.jsx | | └─
| | ├── AdminDashboard.jsx | | ├── components/ | | ├──
| | ├── Navbar.jsx | | ├── StudentsTable.jsx | | ├──
| | ├── PaymentsPanel.jsx | | ├── AttendancePanel.jsx
```

```

| | | └─ StatsCharts.jsx | | └─ context/ | | |
└─ AuthContext.jsx | | └─ services/ | | └─
api.js | └─ index.css └─ database/ └─
schema.sql

```

Estructura 3.1 — Organización del proyecto

► Creación de base de datos

Se implementó el esquema de base de datos con las tablas principales del sistema. Se utilizó **better-sqlite3** como driver sincrónico para SQLite, lo que simplifica el manejo de transacciones y consultas. A continuación se presentan las sentencias DDL de las tablas principales.

```

CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  correo TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  rol TEXT NOT NULL DEFAULT 'estudiante',
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS estudiantes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  usuario_id INTEGER NOT NULL UNIQUE,
  telefono TEXT,
  nivel_actual TEXT DEFAULT '1',
  estado TEXT DEFAULT 'activo',
  FOREIGN KEY (usuario_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS pagos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  estudiante_id INTEGER NOT NULL,
  nivel TEXT NOT NULL,
  monto REAL NOT NULL,
  estado TEXT DEFAULT 'pendiente',

```



```

        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id) ON
    );

CREATE TABLE IF NOT EXISTS progreso (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    estudiante_id INTEGER NOT NULL,
    nivel TEXT NOT NULL,
    porcentaje INTEGER DEFAULT 0,
    estado TEXT DEFAULT 'en_curso',
    FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id) ON
);

CREATE TABLE IF NOT EXISTS asistencia (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    estudiante_id INTEGER NOT NULL,
    fecha DATE NOT NULL,
    presente INTEGER DEFAULT 0,
    FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id) ON
);

```

Código 3.3 — Esquema SQL de las tablas principales

► Página de inicio

Se desarrolló la página de inicio (Home) como la puerta de entrada al sistema. Incluye secciones informativas sobre la institución, los cursos ofrecidos, beneficios, testimonios y un formulario de contacto. La página utiliza componentes reutilizables de Bootstrap y está completamente responsiva.

```

function Home() {
    return (
        <div className="home-page">
            <Hero
                title="H.M.A.S. Armonía Musical"
                subtitle="Tu camino hacia la excelencia musical"
            />
            <Courses>
                <CourseCard title="Piano" level="Niveles 1-9" />
            </Courses>
        </div>
    );
}

```

```

        <CourseCard title="Guitarra" level="Niveles 1-9" />
        <CourseCard title="Violín" level="Niveles 1-6" />
        <CourseCard title="Canto" level="Niveles 1-6" />
      </Courses>
      <Benefits>
        <BenefitItem icon="music" text="Profesores calificac
        <BenefitItem icon="star" text="Método progresivo" />
        <BenefitItem icon="trophy" text="Certificación ofici
      </Benefits>
      <Testimonials testimonials={testimonios} />
      <FAQ items={preguntas} />
      <Contact />
    </div>
  );
}

```

Código 3.4 — Estructura del componente Home.jsx

► Formularios básicos

Se implementaron los formularios de inicio de sesión y registro con validación en tiempo real, alternancia de visibilidad de contraseña y manejo de errores. Los formularios utilizan el estado local de React para gestionar los campos y mostrar retroalimentación inmediata al usuario.

```

function Login() {
  const [formData, setFormData] = useState({
    correo: '',
    password: ''
  });
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState({});
  const { login } = useContext(AuthContext);

  const validate = () => {
    const err = {};
    if (!formData.correo) err.correo = 'El correo es requerido';
    if (!formData.password) err.password = 'La contraseña es requerida';
    else if (formData.password.length < 6)

```

```

    err.password = 'Mínimo 6 caracteres';
    return err;
  };

const handleSubmit = async (e) => {
  e.preventDefault();
  const err = validate();
  setErrors(err);
  if (Object.keys(err).length === 0) {
    await login(formData.correo, formData.password);
  }
};

return (
  <form onSubmit={handleSubmit}>
    <div className="mb-3">
      <label>Correo electrónico</label>
      <input type="email" className="form-control"
        value={formData.correo}
        onChange={(e) => setFormData({...formData, correo:
      />
      {errors.correo && <div className="text-danger">{errc
    </div>
    <div className="mb-3">
      <label>Contraseña</label>
      <div className="input-group">
        <input type={showPassword ? 'text' : 'password'} c
        value={formData.password}
        onChange={(e) => setFormData({...formData, passw
      />
        <button type="button" className="btn btn-outline-s
          onClick={() => setShowPassword(!showPassword)}>
            {showPassword ? 'Ocultar' : 'Mostrar'}
        </button>
      </div>
      {errors.password && <div className="text-danger">{er
    </div>
    <button type="submit" className="btn btn-primary w-100">
      Iniciar Sesión
    </button>
  </form>
);
}

```


► Registro de datos

Se implementó el endpoint de registro que crea un usuario, su registro de estudiante asociado, el progreso inicial y un registro de pago en una sola transacción. Esto garantiza la consistencia de los datos y evita registros huérfanos en el sistema.

```
// POST /api/auth/register
router.post('/register', async (req, res) => {
  const { nombre, correo, password, telefono, nivel } = req.

  const hash = await bcrypt.hash(password, 10);

  const insertUser = db.prepare(
    'INSERT INTO users (nombre, correo, password, rol) VALUE
  );
  const insertEstudiante = db.prepare(
    'INSERT INTO estudiantes (usuario_id, telefono, nivel_ac
  );
  const insertProgreso = db.prepare(
    'INSERT INTO progreso (estudiante_id, nivel, porcentaje,
  );
  const insertPago = db.prepare(
    'INSERT INTO pagos (estudiante_id, nivel, monto, estado)
  );

  const transaction = db.transaction(() => {
    const userResult = insertUser.run(nombre, correo, hash,
    const userId = userResult.lastInsertRowid;

    const estudianteResult = insertEstudiante.run(userId, te
    const estudianteId = estudianteResult.lastInsertRowid;

    insertProgreso.run(estudianteId, nivel, 'en_curso');
    insertPago.run(estudianteId, nivel, obtenerMonto(nivel),

    return { userId, estudianteId };
  });
```

```
});

const result = transaction();
const token = jwt.sign(
  { id: result.userId, rol: 'estudiante' },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);

res.status(201).json({ token, usuario: { id: result.userId } });
```

Código 4.1 — Endpoint de registro con transacción atómica

► Visualización de datos

Se desarrollaron los endpoints para consultar el dashboard del estudiante y del administrador, así como el componente de tabla de estudiantes con sus datos más relevantes.

```
// GET /api/students/dashboard
router.get('/dashboard', authenticate, (req, res) => {
  const estudiante = db.prepare(`
    SELECT u.nombre, u.correo, e.telefono, e.nivel_actual, e.
    FROM estudiantes e
    JOIN users u ON u.id = e.usuario_id
    WHERE e.usuario_id = ?
  `).get(req.user.id);

  const pagos = db.prepare(`
    SELECT * FROM pagos WHERE estudiante_id = ?
    ORDER BY created_at DESC
  `).all(estudiante.id);

  const progreso = db.prepare(`
    SELECT * FROM progreso WHERE estudiante_id = ?
    ORDER BY nivel ASC
  `).all(estudiante.id);

  const asistencia = db.prepare(`
    SELECT fecha, presente FROM asistencia
```

```

WHERE estudiante_id = ? ORDER BY fecha DESC LIMIT 30
`).all(estudiante.id);

res.json({ estudiante, pagos, progreso, asistencia });
});

```

Código 4.2 — Endpoint del dashboard del estudiante

```

function StudentsTable({ students, onEdit, onDelete }) {
  return (
    <div className="table-responsive">
      <table className="table table-dark table-striped">
        <thead>
          <tr>
            <th>Nombre</th>
            <th>Correo</th>
            <th>Teléfono</th>
            <th>Nivel</th>
            <th>Estado</th>
            <th>Acciones</th>
          </tr>
        </thead>
        <tbody>
          {students.map((s) => (
            <tr key={s.id}>
              <td>{s.nombre}</td>
              <td>{s.correo}</td>
              <td>{s.telefono}</td>
              <td>Nivel {s.nivel_actual}</td>
              <td>
                <span className={`badge ${s.estado} === 'acti
                  {s.estado}
                </span>
              </td>
              <td>
                <button className="btn btn-sm btn-warning me
                  Editar
                </button>
                <button className="btn btn-sm btn-danger" or
                  Eliminar
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

```

        </tr>
      )})
    </tbody>
  </table>
</div>
);
}

```

Código 4.3 — Componente `StudentsTable.jsx` con tabla de estudiantes

► Edición de registros

Se implementó la funcionalidad de edición de pagos con lógica de auto-promoción: cuando un administrador aprueba un pago, el sistema automáticamente registra el progreso del estudiante y, si corresponde, lo promueve al siguiente nivel.

```

// PUT /api/admin/payments/:id
router.put('/payments/:id', authorizeAdmin, (req, res) => {
  const { id } = req.params;
  const { estado } = req.body;

  const pago = db.prepare('SELECT * FROM pagos WHERE id = ?')
  if (!pago) return res.status(404).json({ error: 'Pago no encontrado' });

  db.prepare('UPDATE pagos SET estado = ? WHERE id = ?').run(estado, id);

  // Auto-promoción al siguiente nivel
  if (estado === 'aprobado') {
    const estudiante = db.prepare(
      'SELECT * FROM estudiantes WHERE id = ?'
    ).get(pago.estudiante_id);

    const nivelActual = parseInt(estudiante.nivel_actual);
    const nuevoNivel = Math.min(nivelActual + 1, 9);

    db.prepare('UPDATE estudiantes SET nivel_actual = ? WHERE id = ?')
      .run(String(nuevoNivel), pago.estudiante_id);

    db.prepare('UPDATE progreso SET estado = ? WHERE estudiante_id = ?')
      .run('completado', pago.estudiante_id);
  }
});

```



```

        .run('completado', pago.estudiante_id, pago.nivel);

    db.prepare(
        'INSERT INTO progreso (estudiante_id, nivel, porcentaje)
    ).run(pago.estudiante_id, String(nuevoNivel), 'en_curso'
    }

    res.json({ message: 'Pago actualizado', pagoId: id });
});

```

Código 4.4 — Endpoint de edición de pagos con auto-promoción

► Eliminación de datos

Se implementó la eliminación de estudiantes con borrado en cascada de todos sus registros asociados. El frontend muestra un diálogo de confirmación antes de proceder con la eliminación para evitar pérdidas accidentales de información.

```

// DELETE /api/admin/students/:id
router.delete('/students/:id', authorizeAdmin, (req, res) =>
    const { id } = req.params;

    const estudiante = db.prepare('SELECT * FROM estudiantes WHERE id = ?').run(id)
    if (!estudiante) return res.status(404).json({ error: 'Estudiante no encontrado' });

    // CASCADE elimina pagos, progreso, asistencia automáticamente
    db.prepare('DELETE FROM estudiantes WHERE id = ?').run(id)
    db.prepare('DELETE FROM users WHERE id = ?').run(estudiante.id)

    res.json({ message: 'Estudiante eliminado correctamente' });
});

```

Código 4.5 — Endpoint de eliminación de estudiantes

```

const handleDelete = (id) => {
    if (window.confirm('¿Está seguro de eliminar este estudiante?')) {

```

```

    api.delete(`/admin/students/${id}`)
      .then(() => {
        setStudents(students.filter((s) => s.id !== id));
        toast.success('Estudiante eliminado correctamente');
      })
      .catch(() => toast.error('Error al eliminar el estudiante'));
  }
};

```

Código 4.6 — Confirmación de eliminación en el frontend

► Búsquedas básicas

Se agregaron filtros de búsqueda en tiempo real para las tablas de estudiantes y pagos. Los filtros permiten buscar por nombre, correo electrónico y estado, mejorando significativamente la experiencia de usuario al localizar registros específicos.

```

// StudentsTable.jsx - Filtros de búsqueda
const [search, setSearch] = useState('');

const filteredStudents = students.filter((s) => {
  const term = search.toLowerCase();
  return (
    s.nombre.toLowerCase().includes(term) ||
    s.correo.toLowerCase().includes(term)
  );
});

// PaymentsPanel.jsx - Filtros de búsqueda
const [statusFilter, setStatusFilter] = useState('todos');
const [searchTerm, setSearchTerm] = useState('');

const filteredPayments = payments.filter((p) => {
  const matchStatus = statusFilter === 'todos' || p.estado === statusFilter;
  const matchSearch = !searchTerm ||
    p.nombre.toLowerCase().includes(searchTerm.toLowerCase());
  return matchStatus && matchSearch;
});

```


► Inicio de sesión

Se implementó el sistema de autenticación con JWT (JSON Web Tokens) y bcrypt para el hashing de contraseñas. El AuthContext de React maneja el estado global de autenticación y el interceptor de Axios adjunta automáticamente el token a cada petición.

```
// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { correo, password } = req.body;

  const user = db.prepare('SELECT * FROM users WHERE correo = ?').get(correo);
  if (!user) return res.status(401).json({ error: 'Credenciales inválidas' });

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ error: 'Credenciales inválidas' });

  const token = jwt.sign(
    { id: user.id, rol: user.rol },
    process.env.JWT_SECRET,
    { expiresIn: '7d' }
  );

  res.json({
    token,
    usuario: { id: user.id, nombre: user.nombre, correo: user.correo },
  });
});
```

Código 5.1 — Endpoint de inicio de sesión con bcrypt y JWT

```
// AuthContext.jsx
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
```

```

const [token, setToken] = useState(localStorage.getItem('token'));

useEffect(() => {
  if (token) {
    api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
    api.get('/auth/verify').then((res) => {
      setUser(res.data.usuario);
    }).catch(() => {
      localStorage.removeItem('token');
      setToken(null);
    }).finally(() => setLoading(false));
  } else {
    setLoading(false);
  }
}, [token]);

const login = async (correo, password) => {
  const res = await api.post('/auth/login', { correo, password });
  localStorage.setItem('token', res.data.token);
  api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
  setUser(res.data.usuario);
  setToken(res.data.token);
};

const logout = () => {
  localStorage.removeItem('token');
  delete api.defaults.headers.common['Authorization'];
  setUser(null);
  setToken(null);
};

return (
  <AuthContext.Provider value={{ user, login, logout, loading }}>
    {children}
  </AuthContext.Provider>
);
};

// Axios interceptor for JWT
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      delete api.defaults.headers.common['Authorization'];
    }
  }
);

```

```

        window.location.href = '/login';
    }
    return Promise.reject(error);
}
);

```

Código 5.2 — AuthContext.jsx con manejo de JWT e interceptor Axios

► Roles de usuarios

Se implementó un sistema de control de acceso basado en roles (RBAC) con dos niveles: **admin** y **estudiante**. Los middlewares de autenticación y autorización protegen las rutas del backend, mientras que los componentes del frontend muestran u ocultan elementos según el rol del usuario.

```

// middleware/auth.js
const authenticate = (req, res, next) => {
    const header = req.headers.authorization;
    if (!header) return res.status(401).json({ error: 'Token r

    const token = header.split(' ')[1];
    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = decoded;
        next();
    } catch {
        res.status(401).json({ error: 'Token inválido' });
    }
};

// middleware/admin.js
const authorizeAdmin = (req, res, next) => {
    if (req.user.rol !== 'admin') {
        return res.status(403).json({ error: 'Acceso denegado' }
    }
    next();
};

// ProtectedRoute.jsx

```

```

function ProtectedRoute({ children, adminOnly }) {
  const { user, loading } = useContext(AuthContext);

  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (adminOnly && user.rol !== 'admin')
    return <Navigate to="/dashboard" replace />;

  return children;
}

// Navbar.jsx - Role-based links
const navLinks = user?.rol === 'admin'
  ? [
    { label: 'Estudiantes', path: '/admin/students' },
    { label: 'Pagos', path: '/admin/payments' },
    { label: 'Asistencia', path: '/admin/attendance' },
    { label: 'Reportes', path: '/admin/reports' },
  ]
  : [
    { label: 'Mi Perfil', path: '/dashboard' },
    { label: 'Mis Pagos', path: '/payments' },
    { label: 'Mi Progreso', path: '/progress' },
  ];

```

Código 5.3 — Middleware de autenticación, autorización y componentes de protección de rutas

► Reportes

Se desarrollaron endpoints de estadísticas con consultas agregadas que alimentan los gráficos del dashboard administrativo. Los reportes incluyen ingresos por mes, distribución de niveles, asistencia promedio y estado de pagos. Se utilizó Recharts para la visualización de datos.

```

// GET /api/admin/stats
router.get('/stats', authorizeAdmin, (req, res) => {
  const totalStudents = db.prepare(
    'SELECT COUNT(*) as total FROM estudiantes WHERE estado

```

```

    ).get('activo');

    const paymentsByStatus = db.prepare(`
      SELECT estado, COUNT(*) as count, SUM(monto) as total
      FROM pagos GROUP BY estado
    `).all();

    const studentsByLevel = db.prepare(`
      SELECT nivel_actual, COUNT(*) as count
      FROM estudiantes WHERE estado = ?
      GROUP BY nivel_actual ORDER BY nivel_actual
    `).get('activo');

    const monthlyIncome = db.prepare(`
      SELECT strftime('%Y-%m', created_at) as mes,
        SUM(monto) as ingresos
      FROM pagos WHERE estado = ?
      GROUP BY mes ORDER BY mes DESC LIMIT 6
    `).all('aprobado');

    const attendanceRate = db.prepare(`
      SELECT ROUND(AVG(presente) * 100, 1) as tasa
      FROM asistencia
    `).get();

    res.json({
      totalStudents: totalStudents.total,
      paymentsByStatus,
      studentsByLevel,
      monthlyIncome,
      attendanceRate: attendanceRate.tasa
    });
  });
}

```

Código 5.4 — Endpoint de estadísticas con consultas agregadas

```

// StatsCharts.jsx
import { BarChart, Bar, PieChart, Pie, Cell, XAxis, YAxis, T

const COLORS = ['#c9a84c', '#e74c3c', '#2ecc71', '#3498db',

function StatsCharts({ stats }) {

```



```

return (
  <div className="row">
    <div className="col-md-6 mb-4">
      <h5>Ingresos Mensuales</h5>
      <ResponsiveContainer width="100%" height={300}>
        <BarChart data={stats.monthlyIncome}>
          <XAxis dataKey="mes" stroke="#888" />
          <YAxis stroke="#888" />
          <Tooltip />
          <Bar dataKey="ingresos" fill="#c9a84c" radius={10} />
        </BarChart>
      </ResponsiveContainer>
    </div>
    <div className="col-md-6 mb-4">
      <h5>Estado de Pagos</h5>
      <ResponsiveContainer width="100%" height={300}>
        <PieChart>
          <Pie
            data={stats.paymentsByStatus}
            dataKey="count"
            nameKey="estado"
            cx="50%" cy="50%" outerRadius={100}
            label={({value, name, color}) => (
              <div>
                <div>{value}</div>
                <div>{name}</div>
                <div>{color}</div>
              </div>
            )}
          />
          <Tooltip />
          <Legend />
        </PieChart>
      </ResponsiveContainer>
    </div>
  </div>
);
}

```

Código 5.5 — Componente StatsCharts.jsx con gráficos de Recharts

► Diseño responsivo

Se implementó un diseño completamente responsivo utilizando Bootstrap 5 y CSS personalizado. La barra de navegación se adapta a dispositivos móviles con un menú hamburguesa, y se definieron variables CSS para mantener una paleta de colores consistente en toda la aplicación.

```
// Navbar.jsx - Hamburger menu
function Navbar() {
  const { user, logout } = useContext(AuthContext);
  const [expanded, setExpanded] = useState(false);

  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-c
      <div className="container">
        <Link className="navbar-brand" to="/">
          H.M.A.S. Armonía Musical
        </Link>
        <button className="navbar-toggler"
          onClick={() => setExpanded(!expanded)}
          aria-controls="navbarNav" aria-expanded={expanded}
        >
          <span className="navbar-toggler-icon"></span>
        </button>
        <div className={`collapse navbar-collapse ${expanded
          id="navbarNav">
          <div className="navbar-nav ms-auto">
            {user ? (
              <
                <NavLink to={user.rol === 'admin' ? '/admin'
                  Dashboard
                </NavLink>
                <button className="btn btn-outline-light btr
                  Cerrar Sesión
                </button>
              </>
            ) : (
              <
                <NavLink to="/login" className="nav-link">Ir
                <NavLink to="/register" className="nav-link"
              </>
            )
          </div>
        </div>
      </div>
    )}
```

```

        </div>
      </div>
    </div>
  </nav>
);
}

```

Código 5.6 — Navbar.jsx con menú hamburguesa responsivo

```

/* index.css - Variables globales y estilos base */
:root {
  --primary: #c9a84c;
  --primary-dark: #a8882e;
  --bg-dark: #0a0a14;
  --bg-card: #1a1a30;
  --bg-input: #232340;
  --text-primary: #f0f0ff;
  --text-secondary: #b0b0cc;
  --gradient-gold: linear-gradient(135deg, #c9a84c, #e8c84a);
  --gradient-dark: linear-gradient(135deg, #0a0a14, #1a1a3e);
  --shadow-gold: 0 4px 20px rgba(201, 168, 76, 0.3);
}

body {
  background: var(--bg-dark);
  color: var(--text-primary);
  font-family: 'Inter', sans-serif;
}

/* Animaciones */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes slideIn {
  from { opacity: 0; transform: translateX(-20px); }
  to { opacity: 1; transform: translateX(0); }
}

.fade-in { animation: fadeIn 0.4s ease-out; }
.slide-in { animation: slideIn 0.3s ease-out; }

```

```

/* Cards con vidrio esmerilado */
.card-glass {
  background: rgba(26, 26, 48, 0.8);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(201, 168, 76, 0.2);
  border-radius: 16px;
  box-shadow: 0 4px 24px rgba(0, 0, 0, 0.3);
  transition: transform 0.2s, box-shadow 0.2s;
}

.card-glass:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-gold);
}

/* Botones personalizados */
.btn-gold {
  background: var(--gradient-gold);
  color: #0a0a14;
  font-weight: 600;
  border: none;
  border-radius: 8px;
  padding: 10px 24px;
  transition: transform 0.2s, box-shadow 0.2s;
}

.btn-gold:hover {
  transform: scale(1.02);
  box-shadow: var(--shadow-gold);
  color: #0a0a14;
}

```

Código 5.7 — index.css con variables CSS, gradientes y animaciones

► Pruebas funcionales

Se realizaron pruebas funcionales de extremo a extremo (E2E) cubriendo los tres flujos principales del sistema. Las pruebas se ejecutaron manualmente siguiendo los casos de prueba definidos y se documentaron los resultados obtenidos.

Flujo	Descripción	Pasos	Resultado
Flujo 1: Registro e inicio de sesión	Un nuevo estudiante se registra, inicia sesión y accede a su dashboard.	1. Acceder a /register 2. Completar formulario con datos válidos 3. Enviar y recibir token 4. Redirigir al dashboard 5. Cerrar sesión 6. Iniciar sesión con credenciales	✓ Éxito
Flujo 2: Pago y graduación	El administrador aprueba un pago, el estudiante avanza de nivel hasta completar todos.	1. Admin inicia sesión 2. Aprueba pago pendiente 3. Verificar auto-promoción de nivel 4. Repetir hasta nivel 9 5. Ver progreso como "completado"	✓ Éxito
Flujo 3: Asistencia y reportes	Se registra asistencia y se verifican los	1. Admin registra asistencia de estudiantes	✓ Éxito

Flujo	Descripción	Pasos	Resultado
	reportes generados.	2. Consulta dashboard de reportes 3. Verificar gráficos de asistencia 4. Verificar ingresos mensuales 5. Exportar visualización	

Tabla 5.1 — Pruebas funcionales E2E realizadas

Los tres flujos se completaron satisfactoriamente, confirmando que el sistema cumple con los requerimientos funcionales definidos en la etapa de análisis. Se identificaron oportunidades de mejora en la experiencia de usuario que serán abordadas en iteraciones futuras.

► Diagrama de casos de uso

El diagrama de casos de uso presenta los actores del sistema (**Estudiante** y **Administrador**) con todas las funcionalidades disponibles para cada uno. El sistema se delimita dentro del rectángulo de frontera «Sistema H.M.A.S.» que agrupa todos los casos de uso.

```

graph TD
    subgraph "Sistema H.M.A.S."
        UC1[Ver Dashboard]
        UC2[Subir Comprobante]
        UC3[Ver Estado Cuenta]
        UC4[Ver Asistencia]
        UC5[Ver Progreso]
        UC6[Gestionar Estudiantes]
        UC7[Gestionar Pagos]
        UC8[Tomar Asistencia]
        UC9[Ver Reportes]
        UC10[Gestionar Progreso]
        UC11[Registrar Usuarios]
    end

    EST[Estudiante]
    ADM[Administrador]

    EST --> UC1
    EST --> UC2
    EST --> UC3
    EST --> UC4
    EST --> UC5

    ADM --> UC6
    ADM --> UC7
    ADM --> UC8
    ADM --> UC9
    ADM --> UC10
    ADM --> UC11

    style EST fill:#c9a84c,color:#0a0a14,stroke:#c9a84c,stroke-width:2px
    style ADM fill:#1a2a5e,color:#fff,stroke:#c9a84c,stroke-width:2px

```

Figura A.1 — Diagrama de casos de uso del sistema H.M.A.S. Armonía Musical

El **Estudiante** tiene acceso a 5 funcionalidades centradas en la consulta de su información personal, académica y financiera. El **Administrador** tiene acceso a 6 funcionalidades que abarcan la gestión completa de estudiantes, pagos, asistencia, progreso y reportes, además del registro de nuevos usuarios en el sistema.

► Diagrama de flujo de usuario

El diagrama de flujo de usuario muestra el recorrido completo que un visitante puede seguir desde que llega a la página de inicio hasta que interactúa con las funcionalidades del sistema según su rol. Incluye las rutas de registro, inicio de sesión y cierre de sesión.

flowchart TD

START[Visitante] --> HOME[Home]

HOME -->|Registrarse| REG[Register]

HOME -->|Iniciar Sesión| LOGIN[Login]

LOGIN -->|Admin| AD[AdminDashboard]

LOGIN -->|Estudiante| SD[StudentDashboard]

REG -->|Registro exitoso| SD

SD --> VP[View Profile]

SD --> VPR[View Progress]

SD --> MP[Make Payment]

SD --> VA[View Attendance]

AD --> MS[Manage Students]

AD --> MPA[Manage Payments]

AD --> TA[Take Attendance]

AD --> VR[View Reports]

MS -->|Editar| MS

MS -->|Eliminar| MS

MPA -->|Aprobar| MPA

VP -->|Cerrar sesión| LOGOUT[Logout]

VPR --> LOGOUT

MP --> LOGOUT

VA --> LOGOUT

MS --> LOGOUT

MPA --> LOGOUT

TA --> LOGOUT

VR --> LOGOUT

LOGOUT --> HOME

style START fill:#c9a84c,color:#0a0a14,stroke:#c9a84c

style HOME fill:#1a2a5e,color:#fff,stroke:#c9a84c

style AD fill:#1a2a5e,color:#fff,stroke:#c9a84c

```
style SD fill:#1a2a5e,color:#fff,stroke:#c9a84c
style LOGOUT fill:#e74c3c,color:#fff,stroke:#c0392b
```

Figura A.2 — Diagrama de flujo de usuario del sistema H.M.A.S. Armonía Musical

El flujo comienza con el **Visitante** que llega a la página de inicio. Desde allí puede registrarse o iniciar sesión. Dependiendo del rol, es redirigido al **AdminDashboard** o al **StudentDashboard**. Cada dashboard ofrece las funcionalidades correspondientes y desde cualquier pantalla el usuario puede cerrar sesión para volver a la página principal.